

✓ Exercises XP : Evaluating LLMs for Summarization

✓ What you will learn

- Hands-on evaluation for summarization: accuracy vs. ROUGE.
- Strengths/weaknesses of metrics and model size comparisons.
- Using Hugging Face `transformers` + `evaluate` for quick experiments.
- Data loading, sampling, preprocessing, and debugging model outputs.

Create: evaluation scripts, comparison tables, custom metrics, and short analyses.

```
!pip -q install rouge_score==0.1.2 evaluate datasets transformers accelerate nltk --q
```

```
import nltk
nltk.download('punkt')
nltk.download('punkt_tab')
```

```
Preparing metadata (setup.py) ... done
84.1/84.1 kB 3.4 MB/s eta 0:00:00
Building wheel for rouge_score (setup.py) ... done
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
True
```

```
port load_dataset
pd

NN/DailyMail dataset...)
ad_dataset("abisee/cnn_dailymail", "3.0.0", split="test")

raw_dataset.select(range(sample_size))

mplete. Loaded {len(dataset_subset)} samples for evaluation.")
```

```

Loading CNN/DailyMail dataset...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens).
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public mod
warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests
README.md:      15.6k/? [00:00<00:00, 1.13MB/s]

3.0.0/train-00000-of-                                257M/257M [00:03<00:00, 1.29GB/s]
00003.parquet: 100%

3.0.0/train-00001-of-                                257M/257M [00:04<00:00, 252MB/s]
00003.parquet: 100%

3.0.0/train-00002-of-                                259M/259M [00:02<00:00, 1.32GB/s]
00003.parquet: 100%

3.0.0/validation-00000-of-                           34.7M/34.7M [00:00<00:00, 173MB/s]
00001.parquet: 100%

3.0.0/test-00000-of-                                 30.0M/30.0M [00:00<00:00, 150MB/s]
00001.parquet: 100%

Generating train split: 100%                          287113/287113 [00:09<00:00, 35094.11 examples/s]

Generating validation split: 100%                     13368/13368 [00:00<00:00, 50016.49 examples/s]

Generating test split: 100%                          11490/11490 [00:00<00:00, 44551.19 examples/s]

Setup Complete. Loaded 10 samples for evaluation.

```

✓ Part II. Dataset loading and exploration

Preferred dataset: [abisee/cnn_dailymail](https://huggingface.co/datasets/abisee/cnn_dailymail) (map `article` -> `prompt_text`, `highlights` -> `prompt_title`).

- If you have local train/test CSVs with `prompt_text` / `prompt_title`, set the paths below.
- Otherwise, we will auto-sample a small slice from the HF dataset to keep things light.
- Show a couple of rows for a sanity check. If HF download fails, a tiny fallback sample is used.

```

import pandas as pd
from datasets import load_dataset

# Point to your data; leave empty to use the HF cnn_dailymail sample or fallback
train_path = '' # e.g., '/content/train.csv'
test_path = '' # e.g., '/content/test.csv'

fallback = pd.DataFrame([
    {
        'prompt_text': 'The cat sat on the mat and purred loudly while the sun set.',
        'prompt_title': 'Cat rests on mat at sunset'
    },
    {
        'prompt_text': 'Scientists discovered water on the moon, opening new research
        'prompt_title': 'Water found on the moon'
    }
])

```

```

        'prompt_text': 'The local team won the championship after a dramatic final ma
        'prompt_title': 'Local team clinches title'
    },
]
])

def load_and_sample(path, split_name, n):
    if path:
        df = pd.read_csv(path)
    else:
        try:
            hf_split = f"{split_name}[:{max(n, 3)}]"
            print(f"Downloading {split_name} split (v3.0.0) from Hugging Face...")

            ds = load_dataset("abisee/cnn_dailymail", "3.0.0", split=hf_split)

            df = ds.to_pandas()[['article', 'highlights']].rename(
                columns={'article': 'prompt_text', 'highlights': 'prompt_title'}
            )
        except Exception as exc:
            print(f"HF load failed ({exc}); using tiny fallback sample.")
            df = fallback.copy()

    return df.sample(min(n, len(df)), random_state=42).reset_index(drop=True)

train_df = load_and_sample(train_path, 'train', 100)
test_df = load_and_sample(test_path, 'test', 50)

print("\n" + "="*50)
print("### DATA INSPECTION ###")
print(f"Train rows: {len(train_df)} | Test rows: {len(test_df)}")
print(f"Columns: {list(train_df.columns)}")
print("="*50)

print("\n### FIRST TRAINING EXAMPLE ###")
print(f"TEXT: {train_df.loc[0, 'prompt_text'][:250]}...")
print(f"SUMMARY: {train_df.loc[0, 'prompt_title']}")

display(train_df.head(2))

```

Downloading train split (v3.0.0) from Hugging Face...
 Downloading test split (v3.0.0) from Hugging Face...

```

=====
### DATA INSPECTION ###
Train rows: 100 | Test rows: 50
Columns: ['prompt_text', 'prompt_title']
=====

```

```

### FIRST TRAINING EXAMPLE ###
TEXT: SHANGHAI, China -- Championship leader Lewis Hamilton spun out of the Chinese Gr
SUMMARY: Lewis Hamilton fails to clinch world title after spinning out of the Chinese
Briton's lead cut to four points by McLaren team-mate Fernando Alonso .
Alonso finished second in Shanghai behind Ferrari's Kimi Raikkonen .

```

	prompt_text	prompt_title
0	SHANGHAI, China -- Championship leader Lewis Hamilton spun out of the Chinese Grand Prix	Lewis Hamilton fails to clinch world title after spinning out of the Chinese Grand Prix
1	(China) -- China has suspended exports of the Aquila	State-run news agency China orders an investigation into

✓ Part III. Summarization with T5 (implement)

Tasks:

- Write `batch_generator` to yield mini-batches.
- Write `summarize_with_t5` using `t5-small` (or swap sizes) with GPU if available.
- Prefix inputs with "summarize: " and decode with `skip_special_tokens=True`.
- Clear CUDA cache between batches (`torch.cuda.empty_cache()`) and `gc.collect()`.

```
import torch, gc
import pandas as pd
from transformers import AutoTokenizer, T5ForConditionalGeneration
from typing import Iterable, List

def batch_generator(items: List[str], batch_size: int):
    """Yield successive n-sized batches from items."""
    for i in range(0, len(items), batch_size):
        yield items[i : i + batch_size]

def summarize_with_t5(texts: List[str], model_name: str = 't5-small',
                       device = "cuda" if torch.cuda.is_available() else "cpu"):
    print(f"Using device: {device}")

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    model = T5ForConditionalGeneration.from_pretrained(model_name)

    all_summaries = []

    for batch in batch_generator(texts, batch_size):
        prefixed_texts = ["summarize: " + text for text in batch]

        inputs = tokenizer(
            prefixed_texts,
            return_tensors="pt",
            padding=True,
            truncation=True,
            max_length=512
        ).to(device)

        outputs = model.generate(
            inputs["input_ids"],
            max_new_tokens=100,
            num_beams=4,
            early_stopping=True
        )

        decoded_summaries = [
            tokenizer.decode(g, skip_special_tokens=True, clean_up_tokenization_spaces=True)
            for g in outputs
        ]
        all_summaries.extend(decoded_summaries)

        del inputs, outputs
        if device == "cuda":
            torch.cuda.empty_cache()
            gc.collect()

    del model, tokenizer
    gc.collect()

    return all_summaries
```

```
RUN_T5 = True
```

```
if RUN_T5:
```

```

if RUN_T5:
    print("Starting T5 Summarization...")
    train_summaries_t5 = summarize_with_t5(
        train_df['prompt_text'].tolist(),
        model_name='t5-small',
        batch_size=8
    )

    results_df = pd.DataFrame({
        'Article': train_df['prompt_text'].str.slice(0, 200) + "
        'Reference Summary': train_df['prompt_title'],
        'T5-Small Summary': train_summaries_t5
    })

    display(results_df.head())
else:
    print("Skipping T5 generation for speed. Set RUN_T5=True to

```

Starting T5 Summarization...

Using device: cpu

config.json: 1.21k/? [00:00<00:00, 80.3kB/s]

tokenizer_config.json: 2.32k/? [00:00<00:00, 163kB/s]

spiece.model: 100% 792k/792k [00:00<00:00, 13.0MB/s]

tokenizer.json: 1.39M/? [00:00<00:00, 20.8MB/s]

model.safetensors: 100% 242M/242M [00:02<00:00, 178MB/s]

Loading weights: 100% 131/131 [00:00<00:00, 565.21it/s, Materializing param=shared.weight]

generation_config.json: 100% 147/147 [00:00<00:00, 7.81kB/s]

	Article	Reference Summary	T5-Small Summary	
0	SHANGHAI, China -- Championship leader Lewis H...	Lewis Hamilton fails to clinch world title aft...	championship leader Lewis Hamilton spun out of...	
1	(CNN) -- China has suspended exports of the Aq...	State-run news agency: China orders an investi...	china suspends exports of the toys that contain...	
2	(CNN) -- The company was founded in 1985 by se...	The company has become a huge name in communic...	Qualcomm's patent portfolio includes approxima...	
3	ISLAMABAD, Pakistan (CNN) -- Hours after decla...	NEW: President Musharraf orders troops to take...	president pervez Musharraf orders troops to ta...	
4	QUEBEC, Canada -- Third seed Julia Vakulenko w...	Julia Vakulenko has reached her first final on...	third seed Julia Vakulenko will face comeback ...	

✓ Part IV. Accuracy evaluation (toy, likely near zero)

Implement a naive accuracy that checks exact string match between generated and reference summaries. Discuss why this is harsh for free-form text (almost always zero).

```

from typing import List

```

```

def compute_accuracy(preds: List[str], refs: List[str]) -> float:
    """

```

```

Calculates exact string match accuracy.
This is a 'toy' metric for summarization because it requires 100%
character-for-character equality.
"""
if not preds or not refs:
    return 0.0

matches = sum(1 for p, r in zip(preds, refs) if p.strip() == r.strip())

return matches / max(len(refs), 1)

if 'train_summaries_t5' in locals():
    acc = compute_accuracy(train_summaries_t5, train_df['prompt_title'].tolist())
    print(f"Exact-match accuracy: {acc:.4f}")

    print(f"Total Matches: {sum(1 for p, r in zip(train_summaries_t5, train_df['prompt_title'].tolist()) if p.strip() == r.strip())}")
else:
    print("Accuracy skipped (no predictions found).")

```

```

Exact-match accuracy: 0.0000
Total Matches: 0

```

✓ Part V. ROUGE metric implementation

Use `evaluate.load("rouge")` and NLTK sentence tokenizer. Preprocess by joining sentences with newlines for better ROUGE-L.

```

import evaluate
from nltk.tokenize import sent_tokenize
from typing import List

rouge = evaluate.load('rouge')

def normalize_text(text: str) -> str:
    """
    Tokenizes text into sentences and joins them with newlines.
    This helps ROUGE-L calculate the longest common subsequence
    across sentence boundaries more accurately.
    """
    text = str(text).strip()
    sents = sent_tokenize(text)
    return "\n".join(sents)

def compute_rouge_score(preds: List[str], refs: List[str]):
    """
    Computes ROUGE-1, ROUGE-2, and ROUGE-L scores for a list of
    predictions and reference summaries.
    """
    normalized_preds = [normalize_text(p) for p in preds]
    normalized_refs = [normalize_text(r) for r in refs]

    results = rouge.compute(
        predictions=normalized_preds,
        references=normalized_refs,
        use_stemmer=True
    )

    return results

test_preds = ["alpha beta", "", "The cat sat."]
test_refs = ["alpha beta", "reference text", "The cat sat."]

```

```
print("### ROUGE Sanity Check ###")

results = compute_rouge_score(test_preds, test_refs)
for metric, score in results.items():
    print(f"{metric}: {score:.4f}")
```

Downloading builder script: 6.14k/? [00:00<00:00, 644kB/s]

```
### ROUGE Sanity Check ###
rouge1: 0.6667
rouge2: 0.6667
rougeL: 0.6667
rougeLsum: 0.6667
```

✓ Part VI. Understanding ROUGE scores

Experiments to run (describe your findings in a text cell):

- Exact match vs. empty prediction.
- Effect of stemming: e.g., "running" vs. "run".
- N-gram overlap: see how ROUGE-1 vs. ROUGE-2 change with partial overlap.
- Symmetry: swap preds/refs and compare.

```
# 1. Exact match vs. empty prediction:
print("--- 1. Match vs. Empty ---")
print("Exact Match:", compute_rouge_score(["The cat sat."], ["The cat sat."])['rouge1'])
print("Empty Pred: ", compute_rouge_score([""], ["The cat sat."])['rouge1'])

# 2. Effect of stemming (e.g., "running" vs. "run"):
print("\n--- 2. Effect of Stemming ---")
print("Stemming Score:", compute_rouge_score(["The boy is running."], ["The boy runs.

# 3. N-gram overlap (ROUGE-1 vs. ROUGE-2):
print("\n--- 3. N-gram Overlap ---")

scrambled = compute_rouge_score(["Sat the cat mat on."], ["The cat sat on the mat."])
print(f"ROUGE-1 (Unigram): {scrambled['rouge1']:.4f}")
print(f"ROUGE-2 (Bigram): {scrambled['rouge2']:.4f}")

# 4. Symmetry: swap preds/refs and compare:
print("\n--- 4. Symmetry ---")
text_a, text_b = ["A quick fox."], ["The fox was quick."]
print("A -> B:", compute_rouge_score(text_a, text_b)['rouge1'])
print("B -> A:", compute_rouge_score(text_b, text_a)['rouge1'])
```

```
--- 1. Match vs. Empty ---
Exact Match: 1.0
Empty Pred: 0.0

--- 2. Effect of Stemming ---
Stemming Score: 0.8571428571428571

--- 3. N-gram Overlap ---
ROUGE-1 (Unigram): 0.9091
ROUGE-2 (Bigram): 0.2222

--- 4. Symmetry ---
A -> B: 0.5714285714285715
B -> A: 0.5714285714285715
```

✓ Part VII. Comparing small and large models

Goals:

- Generate summaries with `t5-small`, `t5-base`, and `gpt2` (TL;DR style prompt).
- Compute ROUGE for each and store per-row scores.
- Implement `compute_rouge_per_row` to add ROUGE columns to a DataFrame.
- Implement `summarize_with_gpt2` with a TL;DR: prefix and max length guard. Use small batches and low `max_new_tokens` to keep things snappy.

```
from transformers import AutoModelForCausalLM, AutoTokenizer
import torch, gc

def summarize_with_gpt2(texts: List[str], model_name: str = 'gpt
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"Loading {model_name} on {device}...")

    tokenizer = AutoTokenizer.from_pretrained(model_name)
    tokenizer.pad_token = tokenizer.eos_token
    model = AutoModelForCausalLM.from_pretrained(model_name).to(

    all_summaries = []

    for i, batch in enumerate(batch_generator(texts, batch_size)
        print(f" Processing GPT2 batch {i+1}...")
        prompts = [f"{text[:800]}\n\nTL;DR:" for text in batch]

        inputs = tokenizer(prompts, return_tensors="pt", padding

        with torch.no_grad():
            outputs = model.generate(
                **inputs,
                max_new_tokens=max_new_tokens,
                pad_token_id=tokenizer.eos_token_id,
                do_sample=False
            )

        for output in outputs:
            full_text = tokenizer.decode(output, skip_special_to
            summary = full_text.split("TL;DR:")[-1].strip()
            all_summaries.append(summary)

        del inputs, outputs
        if device == "cuda": torch.cuda.empty_cache()
        gc.collect()

    return all_summaries

def compute_rouge_per_row(df: pd.DataFrame, pred_col: str, ref_c
    print(f"Calculating ROUGE-L for {pred_col}...")
    scores = []
    for pred, ref in zip(df[pred_col], df[ref_col]):
        res = compute_rouge_score([str(pred)], [str(ref)])
        scores.append(res['rougeL'])
    return scores

RUN_COMPARE = True

if RUN_COMPARE:
    compare_df = train_df.head(10).copy()

    print("Step 1: Generating with T5-Base...")
```



```
compare_df['t5_base_summary'] = summarize_with_t5(compare_df

print("Step 2: Generating with GPT-2...")
compare_df['gpt2_summary'] = summarize_with_gpt2(compare_df[

if 'train_summaries_t5' in locals():
    compare_df['t5_small_summary'] = train_summaries_t5[:10]

print("Step 3: Computing ROUGE scores...")
compare_df['t5_small_rougeL'] = compute_rouge_per_row(compar
compare_df['t5_base_rougeL'] = compute_rouge_per_row(compar
compare_df['gpt2_rougeL'] = compute_rouge_per_row(compar

print("\n--- Model Comparison Table (ROUGE-L) ---")
display(compare_df[['prompt_title', 't5_small_rougeL', 't5_b
```


Step 1: Generating with T5-Base...

Using device: cpu

Loading weights: 100%

257/257 [00:00<00:00, 708.36it/s, Materializing param=shared.weight]

Step 2: Generating with GPT-2...

Loading gpt2 on cpu...

Loading weights: 100%

148/148 [00:00<00:00, 626.42it/s, Materializing param=transformer.wte.weight]

GPT2LMHeadModel LOAD REPORT from: gpt2

Key	Status	
h.{0...11}.attn.bias	UNEXPECTED	

Part VIII. Comparing all models

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok
A decoder-only architecture is being used, but right-padding was detected! For correct
Implementation: Processing GPT2 batch 1...

- `compare_models` to aggregate average ROUGE across models.
Processing GPT2 batch 2...
A decoder-only architecture is being used, but right-padding was detected! For correct
Implementation: Processing GPT2 batch 3...
- `compare_models_summaries` to show side-by-side summaries. Present the tables and discuss
which model wins and why.
A decoder-only architecture is being used, but right-padding was detected! For correct
Implementation: Processing GPT2 batch 4...

```
import pandas as pd

def compare_models(df: pd.DataFrame, model_cols: list):
    """
    Aggregates ROUGE-L scores into a summary table showing average
    """
    summary_data = []

    for col in model_cols:
        avg_score = df[col].mean()
        summary_data.append({
            "Model Name": col.replace('_rougeL', ''),
            "Average ROUGE-L": round(avg_score, 4)
        })

    return pd.DataFrame(summary_data).sort_values(by="Average RO

def compare_models_summaries(df: pd.DataFrame, pred_cols: list):
    """
    Shows the actual text summaries side-by-side for qualitative
    """
    display_cols = ['prompt_title'] + pred_cols
    return df[display_cols].head(5)

if 'compare_df' in locals():
    rouge_columns = ['t5_small_rougeL', 't5_base_rougeL', 'gpt2_
    prediction_columns = ['t5_small_summary', 't5_base_summary',

    print("### FINAL RESULTS: Aggregated Metrics ###")
    leaderboard = compare_models(compare_df, rouge_columns)
    display(leaderboard)

    print("\n### FINAL RESULTS: Side-by-Side Summary Comparison
    comparison_table = compare_models_summaries(compare_df, pred
    display(comparison_table)
else:
    print("Part VII ran successfully!")
```

FINAL RESULTS: Aggregated Metrics

	Model Name	Average ROUGE-L	
1	t5_base	0.2822	
0	t5_small	0.2373	
2	gpt2	0.1896	

FINAL RESULTS: Side-by-Side Summary Comparison

	prompt_title	t5_small_summary	t5_base_summary	gpt2_summary	
0	Lewis Hamilton fails to clinch world title aft...	championship leader Lewis Hamilton spun out of...	championship leader Lewis Hamilton spun out of...	Hamilton's race was a disaster for McLaren\n\n...	
1	State-run news agency: China orders an investi...	china suspends exports of the toys that contai...	china suspends exports of the Aqua Dots toys c...	The Aqua Dots toys contain a chemical that can...	
2	The company has become a huge name in communic...	Qualcomm's patent portfolio includes approxima...	the company was founded in 1985 by seven commu...	Qualcomm is a global leader in the mobile indu...	
3	NEW: President Musharraf orders troops to take...	president pervez Musharraf orders troops to ta...	president pervez muharraf orders troops to tak...	The government is trying to prevent the countr...	
4	Julia Vakulenko has reached her first final on...	third seed Julia Vakulenko will face comeback ...	third seed Julia vakulenko will face former wo...	Julia Vakulenko will face comeback queen Linds...	

Next steps:

[Generate code with leaderboard](#)

[New interactive sheet](#)

[Generate code with comparison_table](#)

✓ Wrap-up

- Which metrics felt most informative? Why?